

CS 425 / ECE 428

Distributed Systems

Fall 2024

Aishwarya Ganesan
w/ Indranil Gupta (Indy)
The Consistency Spectrum

Consistency Models

Contract between distributed system and application

Dictates what results can the system return for operations

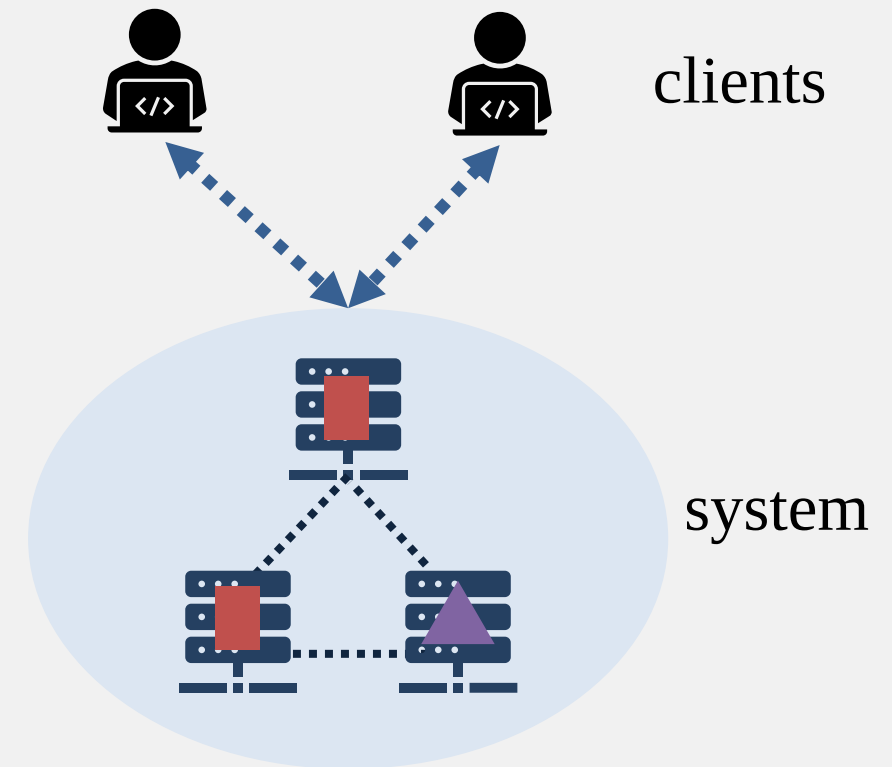
- Concurrent reads and writes by clients on replicas

- Depending on how operations are performed, guarantees offered by the distributed system differ

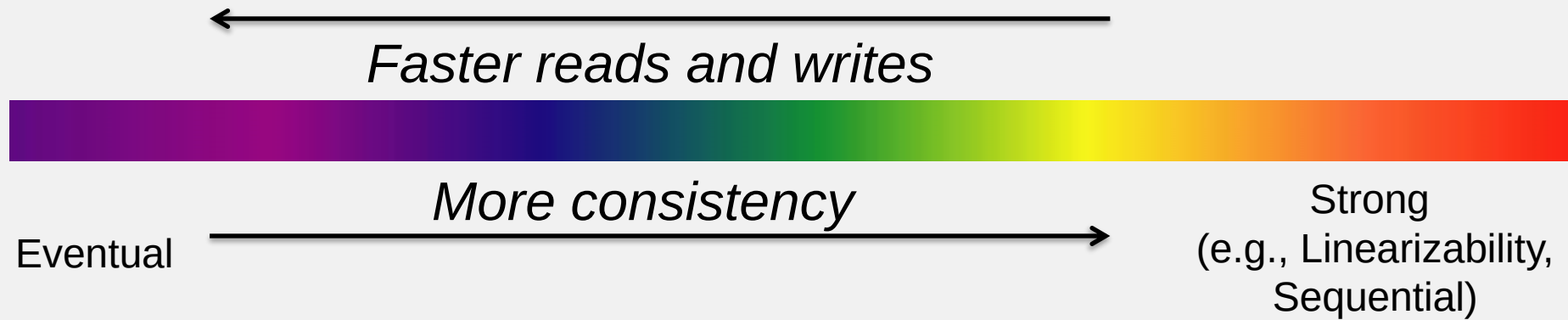
- Consistency model dictates what is permissible

System developers – design/optimize system to meet contract

Application developers – need to understand what contract is for application correctness



Consistency Spectrum



Spectrum Ends: Eventual Consistency

Cassandra offers **Eventual Consistency**

originally from Amazon's Dynamo and LinkedIn's Voldemort

If writes to a key stop, all replicas of key will converge

Reads might see any previous write

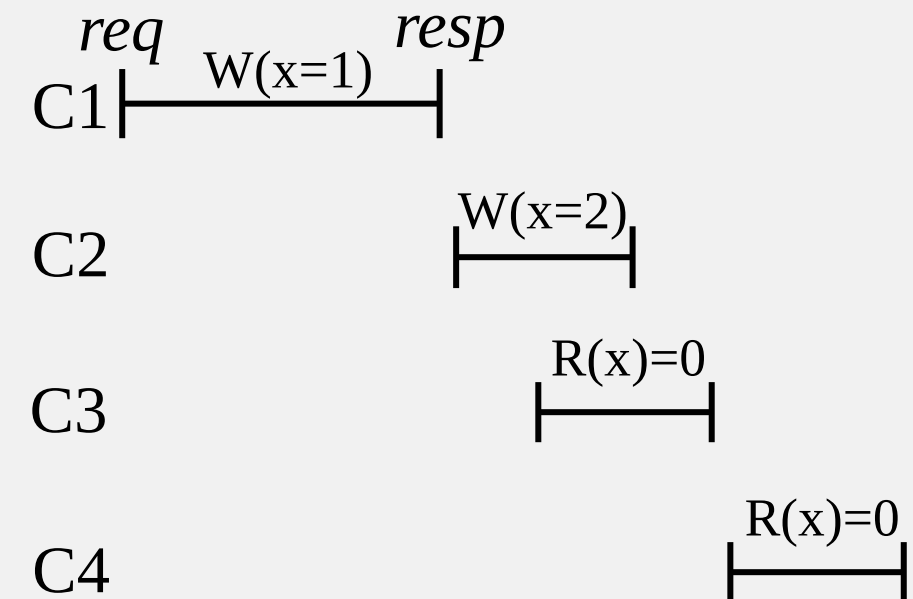
weak guarantee

More **performant** – allows concurrent writes

More **availability** – disconnected operation

Need to resolve conflicts between multiple object versions

Cassandra – latest timestamp wins



Spectrum Ends: Strong Consistency

Linearizability [Herlihy and Wing]

Strong consistency (the **C** in **CAP**)

Intuitively, each operation by a client is visible instantaneously to other clients

instantaneously in real time

All operations are totally ordered in a way that respects real-time order of operations

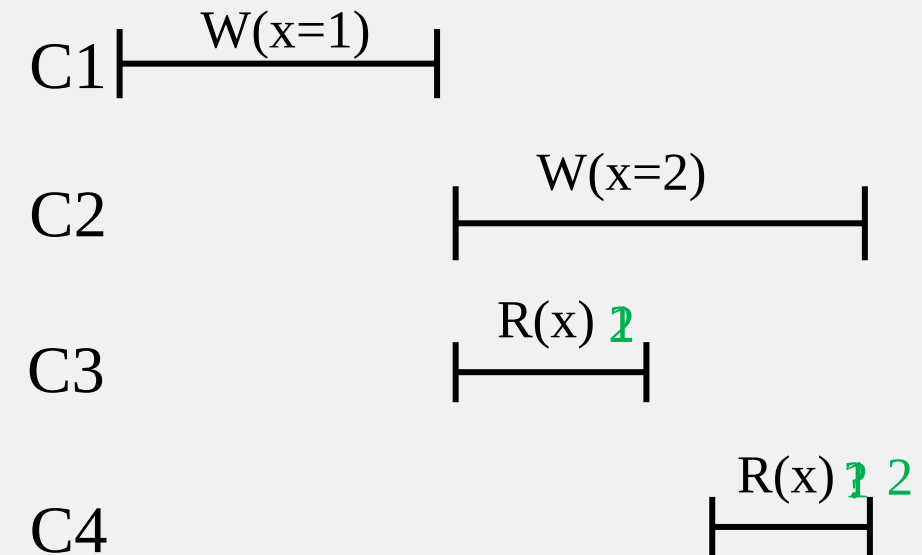
if **A completes before B begins**, A ordered before B

what if A and B are concurrent?

order in a way such that total order is a valid sequential one

Appears to be a single machine (one-copy semantics)

Trades off performance and availability



Spectrum Ends: Strong Consistency Models II

Sequential Consistency (Lamport):

"... the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program."

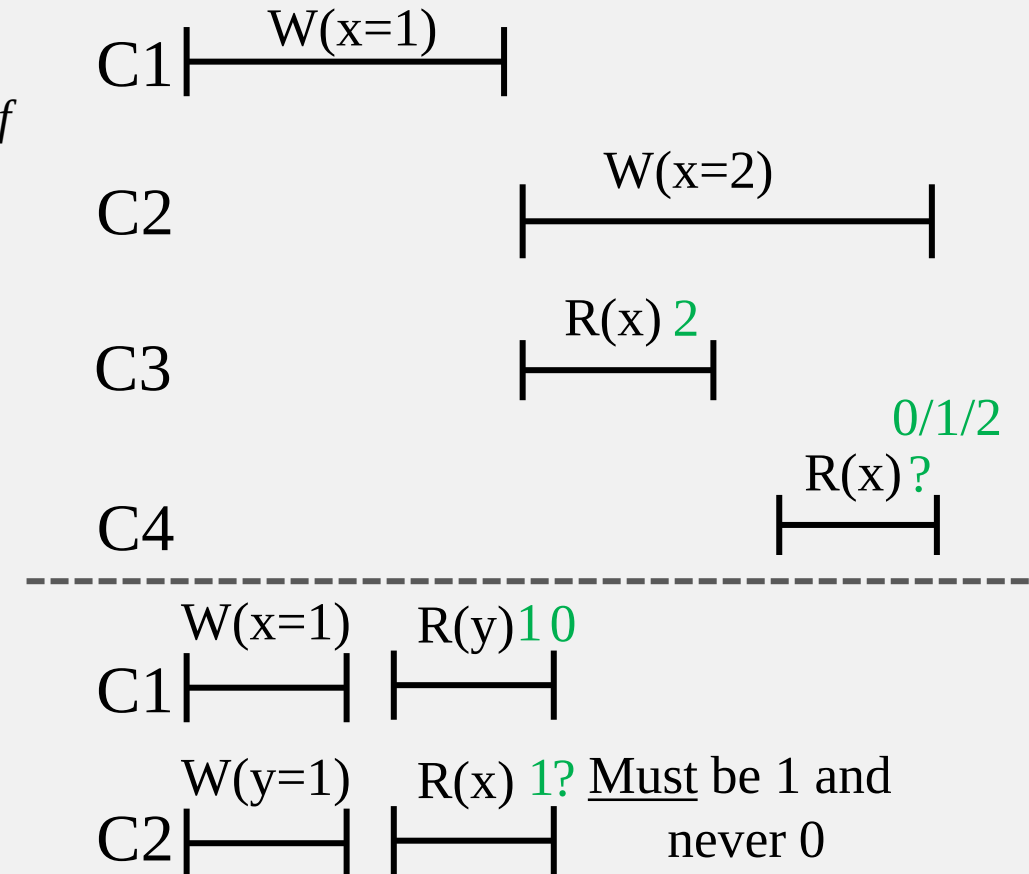
All operations are totally ordered

total order respects local order of operations at clients

Sequential vs linearizability?

sequential is weaker than linearizability (can have stale reads)

but strong compared to many models



Question

Which of the following is a STRONGER consistency model?

- (a) Eventual
- (b) Sequential

Question

Which of the following is a STRONGER consistency model?

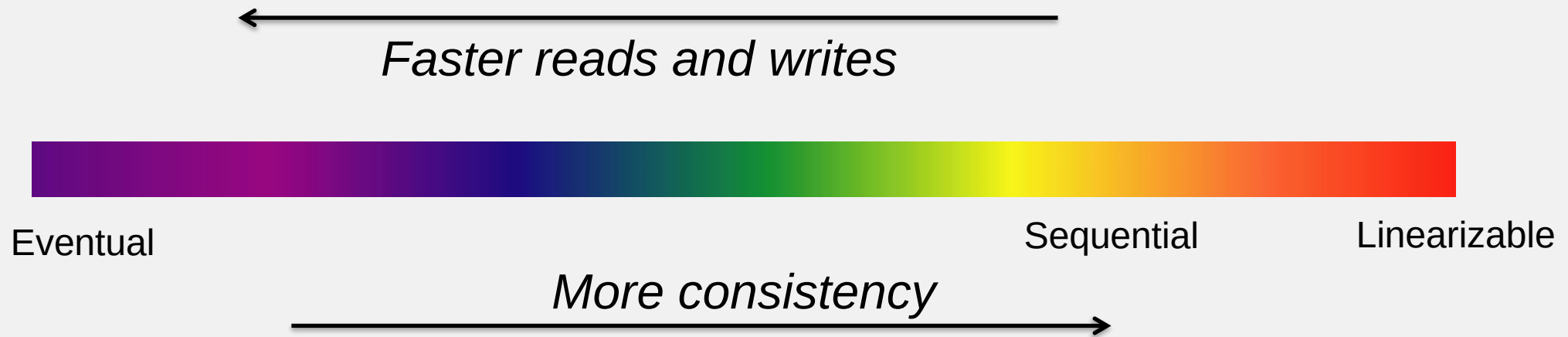
(a) Eventual

(b) Sequential

Answer: (b)

Explanation: Sequential is stronger – it provides a total ordering of operations and respects client's local order

Consistency Spectrum



Linearizability is easy to program against, but can lead to low performance/availability

Eventual consistency is performant and leads to high availability, but hard to program against

There is a variety of options in-between

Causal Consistency

No total order of operations

Available under partitions

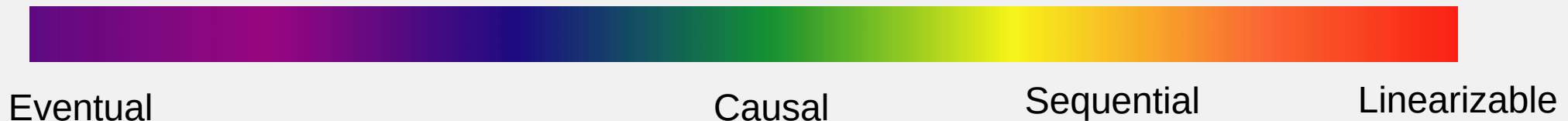
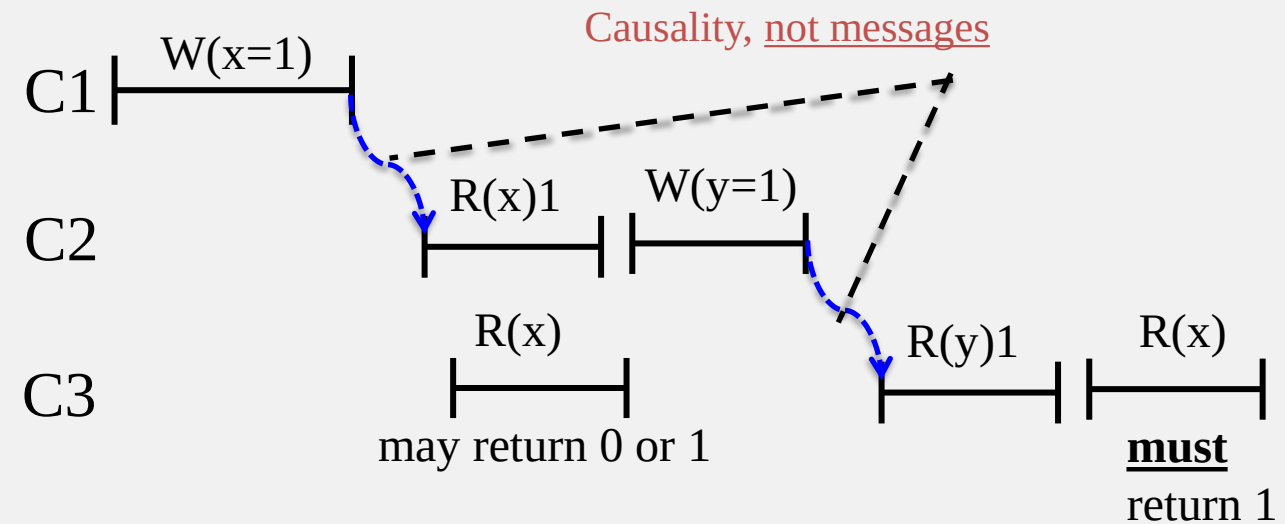
Avoids many inconsistencies that happen with eventual

Operations must respect partial order based on
information flow or causality

Operations from same client are causally related

If client C1 does a write and a read at client C2 sees it,
then these ops are causally related

Transitive



Session-based Consistency Models [Terry et al. 1994]

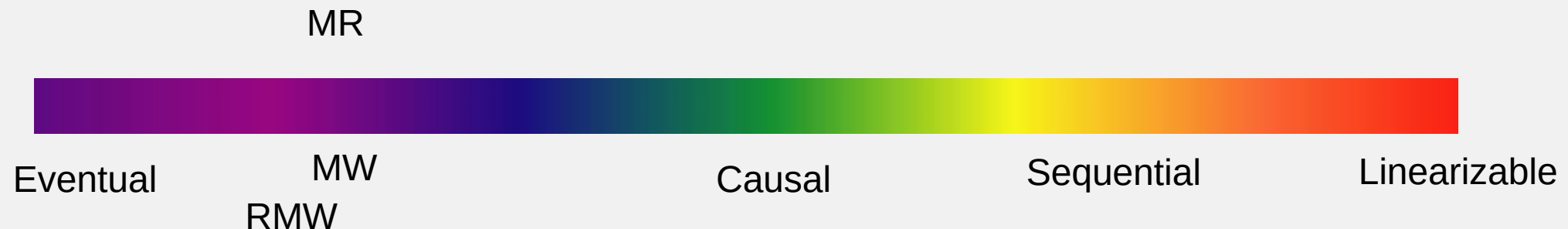
Monotonic reads (MR) – intuitively, reads cannot go back in time

if a client issues read R_1 and then R_2 , R_2 must at least observe the state as of R_1

Monotonic writes (MW) – if a client issues write W_1 and then W_2 , clients must see W_1 before W_2

Read my writes (RMW) – if a client issues W_1 and then R_1 , R_1 must see the effects of W_1

All these models are available in the presence of partitions



Question

Which of the following is TRUE?

- (a) Causal consistency implies monotonic reads
- (b) Causal consistency implies read-my-writes
- (c) Both (a) and (b) are true
- (d) Both (a) and (b) are false

Question

Which of the following is TRUE?

- (a) Causal consistency implies monotonic reads
- (b) Causal consistency implies read-my-writes
- (c) Both (a) and (b) are true**
- (d) Both (a) and (b) are false

Answer: (c)

Explanation: Causal consistency respects client's local order, which ensures that reads do not go back in time and clients always see their own writes.

Newer Consistency Models

CRDTs (Commutative Replicated Data Types): Data structures for which commutated writes give same result [INRIA, France]

- E.g., value == int, and only op allowed is +1

- Guaranteed to eventually converge

- Effectively, servers don't need to worry about consistency and conflict resolution

Red-blue Consistency: Rewrite client transactions to separate ops into red ops vs. blue ops [MPI-SWS Germany and others]

- Blue ops can be executed (commutated) in any order across DCs

- Red ops need to be executed in the same order at each DC

- Composite model: blue ops are eventually consistent and red ops are strongly consistent

Newer Consistency Models (Contd.)

Bounded staleness

Guarantees that data lags at most by K versions or T time units

User or app can set K or T on reads

Probabilistically Bounded Staleness (PBS)

expected bounds on staleness with respect to both versions and time

SLA-style consistency predictions

Question

Which of the following is a CRDT?

- (a) A value of type integer with the only operations allowed being additions and subtractions
- (b) A value of type string that supports appending additional characters to the end of the string

Question

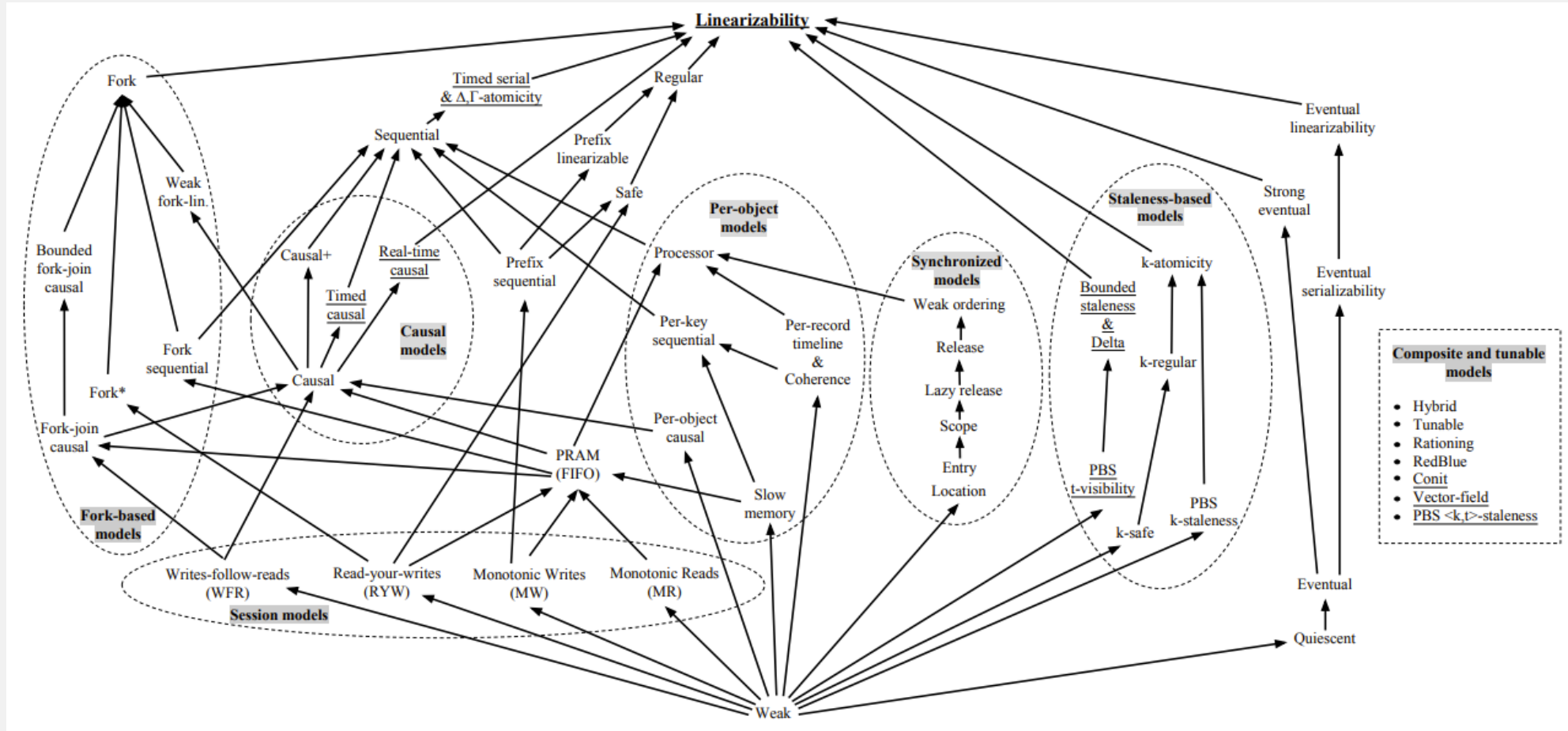
Which of the following is a CRDT?

- (a) A value of type integer with the only operations allowed being additions and subtractions**
- (b) A value of type string that supports appending additional characters to the end of the string

Answer: (a)

Explanation: Additions and subtractions to an integer commute with each other. Appends to a string do not commute.

The Consistency Spectrum



Which Consistency Model should you use?

- Use the lowest consistency model that is “correct” for your application
 - Gets you fastest availability